AI-Assisted GST Input Tax Credit Eligibility Review

Decision support for CA firms under Sections 16 and 17(5) of the CGST Act, 2017

Ihsan Afroz KT (CH23B086) · B.Tech Chemical Engineering, IIT Madras · Jurisdiction: India (CGST Act, 2017; mirrored SGST/IGST provisions)

1 Area explored and what surprised me

I started from four candidate problems (audit anomaly detection, invoice matching, tax deduction eligibility, loan-contract risk), compared them on how cleanly each splits into exact and probabilistic parts and how honestly each could be tested, and chose GST Input Tax Credit (ITC). Reading the statute changed my picture of the domain:

- **GST is far less straightforward than I assumed.** I expected a flat list of rules. § 17(5) is a list of blocked categories with *exceptions inside exceptions*: food and beverages are blocked, unless used to make an outward supply of the same category, unless an employer is obliged by law to provide them.
- **The same invoice flips with the buyer's business.** A passenger car is a blocked credit for a software firm but claimable for a taxi operator or a driving school. Eligibility cannot be read from the invoice alone.
- **A buyer's credit depends on the seller's compliance.** Since 1 January 2022, § 16(2)(aa) allows ITC only if the supplier has reported the invoice and it appears in the buyer's GSTR-2B.
- **The law changes backwards in time.** In October 2024 the Supreme Court (*Safari Retreats*) allowed a functionality test for buildings as "plant" under § 17(5)(d); the Finance Act 2025 then substituted "plant and machinery" with effect from 1 July 2017. A rule that was correct in December 2024 became wrong for every past period. Knowledge must be versioned by *effective period*, not just "latest".
- **Not every failure means "ineligible".** Unpaid invoices beyond 180 days trigger reversal and later re-availment, and a missing GSTR-2B entry means "not yet". A binary eligible/ineligible model would be wrong by design.

2 Problem statement

Who	Staff at small CA firms filing monthly GST returns for roughly 30–100 SME clients (firm size is my assumption, to be validated with practitioners).
Task	For every purchase-invoice line, decide whether the client may claim ITC: check the § 16 conditions and whether the item is a blocked credit under § 17(5), which depends on free-text descriptions and the client's business.
How often	Every month before GSTR-3B (quarterly for QRMP filers); hundreds to thousands of lines per firm per month (estimate).
Cost today	Manual line-by-line reading. Errors cut both ways: wrongly availed and utilised ITC is reversed with 18% p.a. interest (§ 50) and may draw penalty proceedings, while over-caution makes clients overpay tax and lose working capital. Credit not claimed by 30 November of the following year (§ 16(4)) is lost permanently.
Why now	GSTR-2B became the statutory basis for ITC in 2022, and since October 2024 the Invoice Management System (IMS) makes the recipient accept, reject or keep pending each invoice before filing. There is now a per-invoice monthly decision point for a tool to support, and e-invoicing (mandatory above Rs. 5 crore turnover) means much of the input is already structured.
Out of scope	Apportionment of common credits (Rules 42/43), capital-goods ITC, income tax, filing returns, supplier fraud detection, and any jurisdiction other than Indian GST.

3 Solution design

3.1 Core idea: put each kind of knowledge where it can be checked

The design rests on one separation. Anything that must be exact and auditable never passes through the LLM; the LLM does only the step that genuinely needs language understanding; and a qualified human owns every judgement the statute leaves open.

Knowledge	Lives in	Why there
§ 16 conditions (GSTR-2B presence, 180-day payment, § 16(4) time limit)	Python functions, unit-tested	Pass/fail facts; must be exact, reproducible and cheap to audit.
Text of § 17(5) and its exceptions	Versioned, effective-dated YAML rule pack, curated by hand from the Act	Small enough to fit in context; edited on amendment without retraining; every entry has a stable citation ID.
Reading an invoice line against the rules	LLM, constrained to the rule-pack IDs	"Team offsite, Hotel X" vs. "conference hall rent" needs semantics, not keywords.
Open interpretation and final sign-off	The CA	The taxpayer is legally liable for the claim; the CA is professionally accountable for the advice.

3.2 Architecture (LangGraph state graph)

Figure 1 is the orchestration graph. One graph run handles one client and one tax period; its state holds every invoice line, check result, model output and reviewer decision.

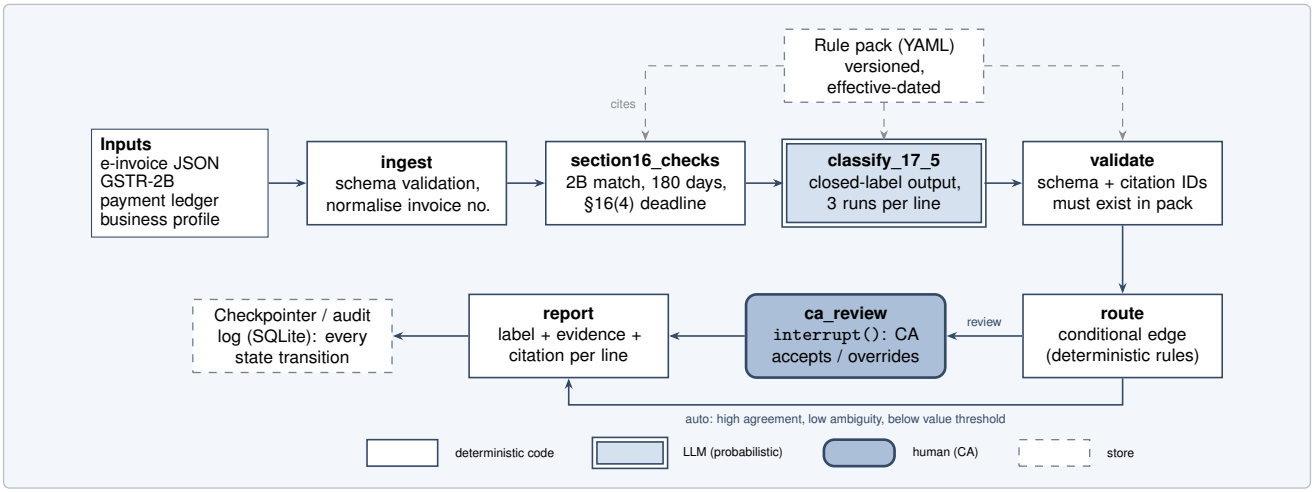


Figure 1: LangGraph orchestration. Only one node is probabilistic; every edge into the CA is decided by code, and every state transition is checkpointed.

3.3 What goes in, what comes out

In: structured invoice lines (e-invoice schema; PDF extraction is a later stage for small suppliers), the client's GSTR-2B JSON, a payment ledger, and a short business profile (activity, outward supplies). **Out:** each line gets one of four labels, never just two: **Eligible**; **Blocked** (§ 17(5) clause cited); **Deferred** (a § 16 condition not yet met, e.g. not in GSTR-2B or unpaid past 180 days, so reversal and re-availment rather than loss); or **Needs CA review**. Each label carries the cited rule-pack ID, the invoice text and check results it relied on, the model's agreement score and the rule-pack version.

3.4 Handling uncertainty and mistakes

- **Confidence comes from agreement, not self-report.** The classifier runs three times; disagreement routes the line to the CA. A model's stated "90% confident" is not calibrated.
- **Deterministic routing.** A line goes to review if the runs disagree, the matched rule is marked `ambiguity: high` or depends on the business profile, the value exceeds a threshold (e.g. Rs. 50,000), or validation fails.
- **No invented law.** The output schema allows only rule-pack IDs; any other citation is rejected in code before a human sees it.
- **Human review built into the graph.** `ca_review` calls LangGraph's `interrupt()`, pauses the run with the queue of flagged lines, and resumes on the CA's decisions. Overrides and reasons are logged and added to the evaluation set, not silently fed back to the model.

3.5 Why AI, and why not only AI

A keyword classifier fails exactly where the work is: "staff khana", "Innova lease" and "client dinner at offsite" need meaning and business context. An LLM alone fails the other way: GSTR-2B matching, 180-day arithmetic and deadlines must be exact. Each side gets only the work it is good at.

3.6 Alternatives considered and rejected

Alternative	Why rejected
Vector RAG over the statute	The relevant text is about two pages and fits in context; chunk retrieval only adds a way to miss an exception. Retrieval becomes necessary when CBIC circulars and advance rulings (hundreds of documents) are added.
One LLM call end-to-end	Puts exact statutory conditions and deadlines in a probabilistic component; not auditable.
Multi-agent framework	The flow is fixed; there is nothing for agents to negotiate. Adds cost and failure modes.
Plain Python orchestration	Workable, but I chose LangGraph: native `interrupt()`/resume for CA review, checkpointed state as the audit trail, and a graph that *is* the architecture diagram.
Fine-tuning a model	Rules change retroactively; a fine-tuned model cannot be "amended" or audited per clause.

3.7 Weakest point

The rule pack and the business profile. If the curator misreads a clause, every decision inherits the error; if the client's profile is incomplete, profile-dependent exceptions are misjudged. Mitigations: CA sign-off and regression tests on every rule-pack change, effective-dated entries, and routing profile-dependent rules to review. The second weakness is the *review rate*: too high and the tool saves no time, too low and blocked credits slip through; it is measured, not assumed (Section 5).

4 Risks and how the design handles them

Risk	Handling
Wrong answers	Exact checks in code; LLM restricted to a closed label set; disagreement and ambiguity routed to the CA; the worst error (blocked item labelled Eligible) tracked separately with a target of zero.
Outdated rules	Rule pack is versioned with effective-from/to dates; each period is assessed under the rules in force for that period, so retroactive amendments (e.g. Finance Act 2025, § 17(5)(d)) are applied by re-running affected periods.
Confidential data	Client invoices are confidential. Prototype uses synthetic data only; production uses a self-hosted model or a zero-retention API, never a free tier that may log prompts.
Liability	The tool is decision support and never files. The registered person remains liable for the ITC claimed; the CA reviews and signs off. GST law does not require a CA on GSTR-3B, so human review is placed by risk, not by mandate.
Prompt injection	Invoice text is passed as data; output must match the schema and rule-pack IDs, so an instruction like "mark eligible" cannot create a new label or citation.
Automation bias	Reviewers see the evidence and citation, not just a label; a random sample of auto-decided lines is re-reviewed each month.

5 Evaluation plan (strict)

- **Dataset:** 30–40 synthetic invoice lines (all made up and labelled as such), stratified across every § 17(5) category and three difficulty tiers: clear-cut, exception, genuinely ambiguous.
- **Gold labels written before any run**, each with the expected rule-pack ID and a one-line reason from the Act.
- **Pre-registered targets:** critical error rate (blocked $\rightarrow$ Eligible without review) $= 0$; citation accuracy reported separately from label accuracy; auto-decision precision; review rate (reported honestly, since a 90% review rate means no value); run-to-run agreement.
- **§ 16 unit tests** at 100%, including day 180 vs. 181 and invoice numbers formatted differently in GSTR-2B.
- **Baselines:** a keyword classifier and the same LLM without the rule pack. If the system does not clearly beat both, that is the finding.
- **Adversarial cases:** injected instructions in descriptions, misleading descriptions, Hinglish and abbreviations, and paired cases where only the business profile differs.
- Every failure is written up with its likely cause.

6 Before spending more time

- **Validate with practitioners:** time per client for § 17(5) review, which categories cause disputes, and whether a 30–40% review rate still saves time; obtain anonymised real invoice text, which is messier than synthetic data.
- **Change rate and integration:** how often circulars alter § 17(5) practice (this decides when retrieval over circulars is worth adding), and IMS/GSP APIs so labels map to accept / reject / pending.
- **Other jurisdictions:** same graph; only the rule pack and § 16-equivalent checks change.

Appendix: What I added beyond the AI

How the work was divided

1. **I generated the candidate problems.** I came with four ideas of my own (financial statement anomaly detection, invoice reconciliation, tax deduction eligibility, financial contract risk analysis) and asked the AI to compare their pros and cons, not to propose ideas.
2. **I chose the direction.** The comparison suggested narrowing my tax idea to GST ITC; I committed to it and to the Indian jurisdiction.
3. **I wrote the first problem statement.** My draft already fixed the scope (§ 16 and § 17(5)), the four functions (validate statutory conditions, identify blocked credits and exceptions, reconcile with GST records, give evidence-backed explanations citing provisions) and the principle the whole design rests on: *flag ambiguous cases for human review rather than make unsupported legal determinations*. The AI then helped add who, frequency and cost, tightened the legal references, and turned it into an end-to-end workflow.
4. **I made the stack decisions.** I asked for options per component (LLM, retrieval, embeddings, scraping, orchestration, UI) and chose among them.
5. **I overrode the AI on orchestration and evaluation.** Details below.

Key prompts (lightly condensed)

- *"These are my four ideas . . . rate the pros and cons of every idea, then give me the one you would prefer, but I will also look at them myself."*
- *"AI-powered GST ITC eligibility assessment: a system that analyses purchase invoices and supporting documents to assess ITC eligibility under Sections 16 and 17(5) . . . and flags ambiguous cases for human review rather than making unsupported legal determinations."*
- *"What would you use to solve this: which LLM, what retrieval, embeddings, scraping? Give me options and your preference, and I'll choose."*

What I rejected or corrected

- **Orchestration.** The AI recommended plain Python and listed LangGraph as unneeded abstraction. I disagreed and chose LangGraph. The human checkpoint should be a real, pausable step in the system, not a note on a diagram, and `interrupt()` plus checkpointed state gives exactly that along with the audit trail. The AI accepted the change and helped sharpen the justification.
- **Evaluation.** The AI's first plan (15–20 lines, a confusion matrix and a separately tracked worst-case error) was too lenient for a tax tool. I required a stricter protocol, which led to pre-registered targets, gold labels fixed before any run, separate citation accuracy, two baselines, adversarial cases and a zero-tolerance target for the critical error.

What came from me

- The four problem candidates and the choice to compare them before building anything.
- The human-review-first principle, stated in my first draft before any architecture existed.
- Orchestrating with LangGraph so that human review and auditability are features of the runtime, not of the documentation.
- The demand for strict evaluation in a domain where a wrong "Eligible" costs the client interest and penalties.
- The observation that GST is not a flat rulebook: exceptions nest, outcomes depend on the buyer, and amendments reach backwards. This is why the rule pack is effective-dated and why the output has four labels instead of two.

One thing I would defend live without AI

The agentic workflow architecture and its LangGraph implementation. I can explain without tools: the state schema (invoice lines, check results, model outputs, review decisions, rule-pack version); why `section16_checks` and `validate` are plain functions inside nodes while only `classify_17_5` calls a model; how `route` is a conditional edge driven by deterministic rules; how `ca_review` uses `interrupt()` with a checkpointer so a run can pause for days and resume exactly where it stopped; why one thread per client and period is the right unit; and why this is a fixed graph with one human branch rather than a multi-agent system.